

Deep Deterministic Policy Gradient

Hyelin Choi

Department of Mathematics
Sungkyunkwan University

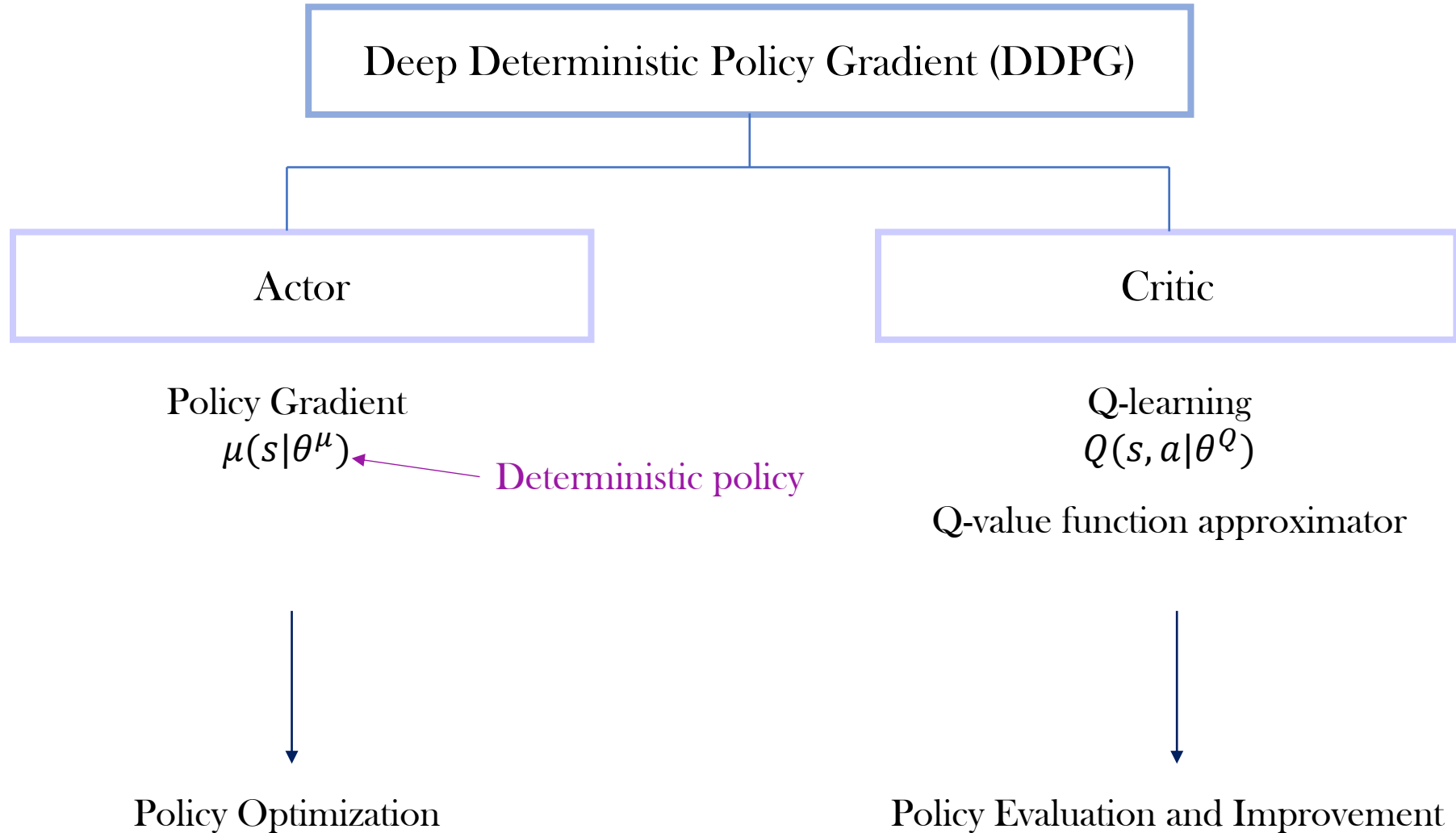
May 7, 2024

Notation

- E : environment
- ρ^π : discounted state visitation distribution for a policy π

$$\rho^\pi(s) = \sum_{t=0}^{\infty} \gamma^t P(s_t = s | \pi)$$

Overview



Actor

The goal is to learn a policy which maximizes

$$J(\theta) = \mathbb{E}_{s \sim \rho^\mu} [Q^\mu(s, a)] = \int_S \rho^\mu(s) Q^\mu(s, a) ds \quad \text{where } a = \mu_\theta(s)$$

Actor

The update scheme is as below:

$$\theta_{k+1} = \theta_k + \alpha \nabla_{\theta} J(\theta_k)$$

where

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \int_S \rho^{\mu}(s) \nabla_a Q(s, a) \Big|_{a=\mu(s|\theta^{\mu})} \nabla_{\theta^{\mu}} \mu(s|\theta^{\mu}) ds \\ &= \mathbb{E}_{s_t \sim \rho^{\beta}} \left[\nabla_a Q(s, a|\theta^Q) \Big|_{s=s_t, a=\mu(s_t)} \nabla_{\theta^{\mu}} \mu(s|\theta^{\mu}) \Big|_{s=s_t} \right] \end{aligned}$$

```
# updating actor
self.actor_optim.zero_grad()
a_loss = -self.critic( (s, w.detach()), self.actor((s, w.detach())) ).mean()
a_loss.backward()
self.actor_optim.step()
```

Q-learning, a commonly used off-policy algorithm, uses the greedy policy

$$\mu(s) = \operatorname{argmax}_a Q(s, a).$$

Critic

We consider function approximators parameterized by θ^Q , which we optimize by minimizing the loss:

$$L(\theta^Q) = \mathbb{E}_{s_t \sim \rho^\beta, a_t \sim \beta, r_t \sim E} \left[(Q(s_t, a_t | \theta^Q) - y_t)^2 \right]$$

where

$$y_t = r(s_t, a_t) + \gamma Q(s_{t+1}, \mu(s_{t+1}) | \theta^Q).$$

Bellman Equation $Q^\mu(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1} \sim E} [r(s_t, a_t) + \gamma Q^\mu(s_{t+1}, \mu(s_{t+1}))]$



Critic

We consider function approximators parameterized by θ^Q , which we optimize by minimizing the loss:

$$L(\theta^Q) = \mathbb{E}_{s_t \sim \rho^\beta, a_t \sim \beta, r_t \sim E} \left[(Q(s_t, a_t | \theta^Q) - y_t)^2 \right]$$

where

$$y_t = r(s_t, a_t) + \gamma Q(s_{t+1}, \mu(s_{t+1}) | \theta^Q).$$

It is possible to learn Q^μ off policy, using transitions which are generated from a different stochastic behavior policy β .



Target policy $\neq$ behavior policy

Critic

We consider function approximators parameterized by θ^Q , which we optimize by minimizing the loss:

$$L(\theta^Q) = \mathbb{E}_{s_t \sim \rho^\beta, a_t \sim \beta, r_t \sim E} \left[(Q(s_t, a_t | \theta^Q) - y_t)^2 \right]$$

where

$$y_t = r(s_t, a_t) + \gamma Q(s_{t+1}, \mu(s_{t+1}) | \theta^Q). \longleftarrow \text{target value}$$

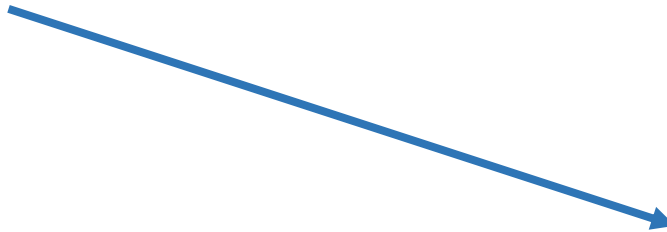
```
y = r + self.discount*isnt*self.critic_target( (s_next, a.detach()), self.actor_target((s_next, a.detach())) ) )

# updating critic
self.critic_optim.zero_grad()
q_pred = self.critic( (s, w.detach(), a.detach()))
c_loss = self.criterion(q_pred, y)
c_loss.backward()
self.critic_optim.step()
```

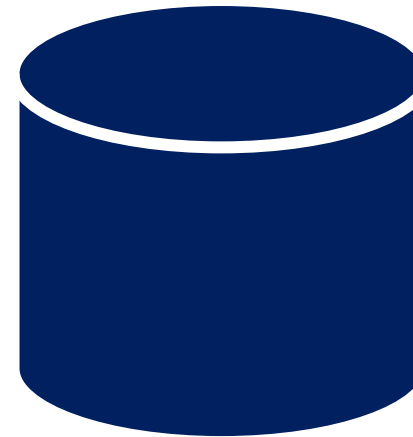
Replay Buffer

DDPG is an off-policy algorithm.

Transition
 (s_i, a_i, r_i, s_{i+1})



$$R = \{(s_1, a_1, r_1, s_2), \\ (s_2, a_2, r_2, s_3), \\ (s_3, a_3, r_3, s_4), \\ \vdots \\ (s_M, a_M, r_M, s_M)\}$$



Replay Buffer

Replay Buffer

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ .

Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$

Initialize replay buffer R

for episode = 1, M **do**

 Initialize a random process $\mathcal{N}$ for action exploration

 Receive initial observation state s_1

for t = 1, T **do**

 Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise

 Execute action a_t and observe reward r_t and observe new state s_{t+1}

 Store transition (s_t, a_t, r_t, s_{t+1}) in R

 Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R

 Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$

 Update critic by minimizing the loss: $L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$

 Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

 Update the target networks:

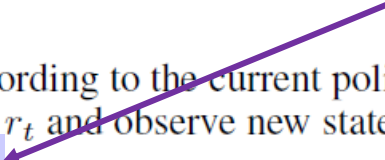
$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$$

$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$$

end for

end for

Replay Buffer



Replay Buffer

```
# Buffer initialization  
self.s_buffer = []  
self.a_buffer = []  
self.r_buffer = []  
self.isnt_buffer = []  
self.s_next_buffer = []  
self.w_buffer = []
```

```
self.buffer = list() # initializes empty list as buffer
```

```
def save_transition(self, state, action, r, is_nonterminal, next_state, prev_action):  
    # self.buffer.append((state, action, r, is_nonterminal, next_state, prev_action))  
    self.s_buffer.append(state)  
    self.a_buffer.append(action)  
    self.r_buffer.append(r)  
    self.isnt_buffer.append(is_nonterminal)  
    self.s_next_buffer.append(next_state)  
    self.w_buffer.append(prev_action)
```

Target Network

We consider function approximators parameterized by θ^Q , which we optimize by minimizing the loss:

$$L(\theta^Q) = \mathbb{E}_{s_t \sim \rho^\beta, a_t \sim \beta, r_t \sim E} \left[\left(Q(s_t, a_t | \theta^Q) - y_t \right)^2 \right]$$

where

$$y_t = r(s_t, a_t) + \gamma Q(s_{t+1}, \mu(s_{t+1}) | \theta^Q). \quad \leftarrow \text{target value}$$



Q update prone to divergence.



Both a target μ' and Q' was required to have stable target in order to consistently train the critic without divergence.

Target Network

We consider function approximators parameterized by θ^Q , which we optimize by minimizing the loss:

$$L(\theta^Q) = \mathbb{E}_{s_t \sim \rho^\beta, a_t \sim \beta, r_t \sim E} \left[(Q(s_t, a_t | \theta^Q) - y_t)^2 \right]$$

where

$$y_t = r(s_t, a_t) + \gamma Q' \left(s_{t+1}, \mu'(s_{t+1} | \theta^{\mu'}) | \theta^{Q'} \right) \longleftarrow \text{fix}$$

$\theta^{Q'}$
 $\theta^{\mu'}$

Target
Networks

θ^Q
 θ^μ

Q Network

- Target Networks, $\theta^{Q'}$ and $\theta^{\mu'}$, are used for calculating the target values.

Target Network

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ .

Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q, \theta^{\mu'} \leftarrow \theta^\mu$ **Hard Update**

Initialize replay buffer R

for episode = 1, M **do**

 Initialize a random process $\mathcal{N}$ for action exploration

 Receive initial observation state s_1

for t = 1, T **do**

 Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise

 Execute action a_t and observe reward r_t and observe new state s_{t+1}

 Store transition (s_t, a_t, r_t, s_{t+1}) in R

 Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R

 Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$

 Update critic by minimizing the loss: $L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$

 Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

 Update the target networks:

$$\begin{aligned} \theta^{Q'} &\leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'} \\ \theta^{\mu'} &\leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'} \end{aligned}$$

Soft Update

$\tau \ll 1 \longrightarrow$

The target values are constrained to change slowly.

end for
end for

Target Network

```
def hard_update(target, source):  
    for target_param, param in zip(target.parameters(), source.parameters()):  
        target_param.data.copy_(param.data)
```

Target Network

```
self.tau = 10e-2
```

```
def soft_update(target, source, tau):  
    for target_param, param in zip(target.parameters(), source.parameters()):  
        target_param.data.copy_(  
            target_param.data * (1.0 - tau) + param.data * tau  
        )
```

Code

1. network
2. ddpg

Code

1. network
2. ddpq

Code (network)

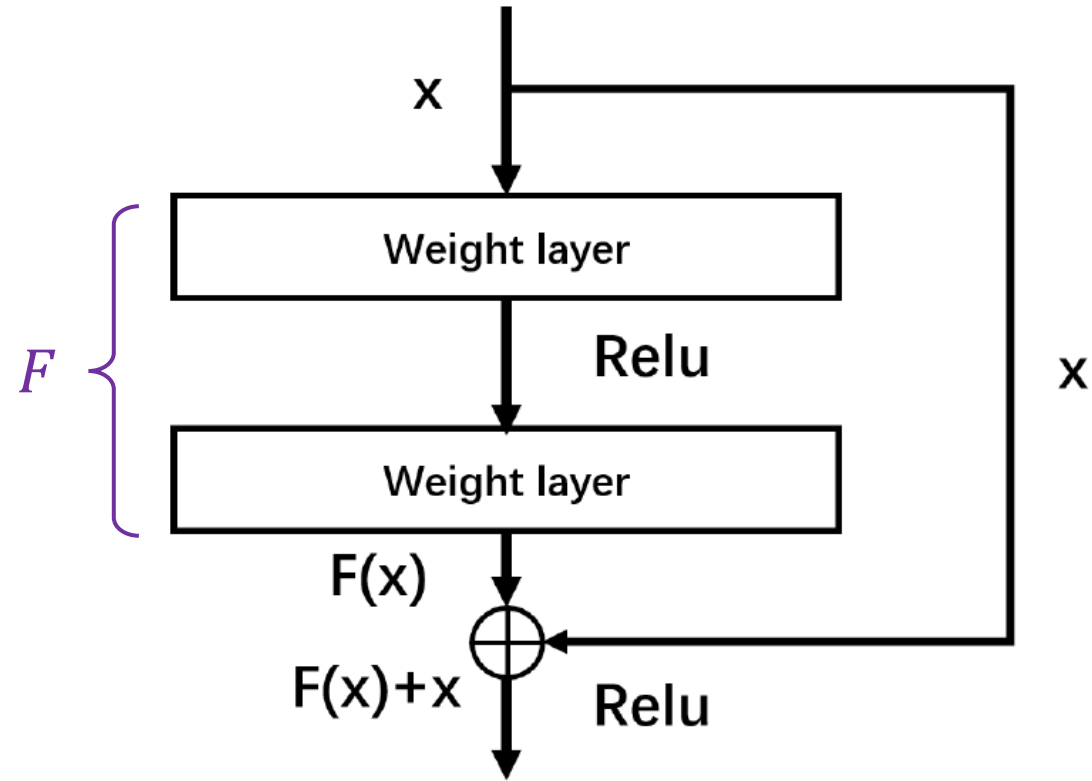


Fig. 2. Residual Block

Code (network)

```
class cnnNetwork(nn.Module):
    def __init__(self, M, L, N):
        super(cnnNetwork, self).__init__()
        self.M = M
        self.L = L
        self.N = N

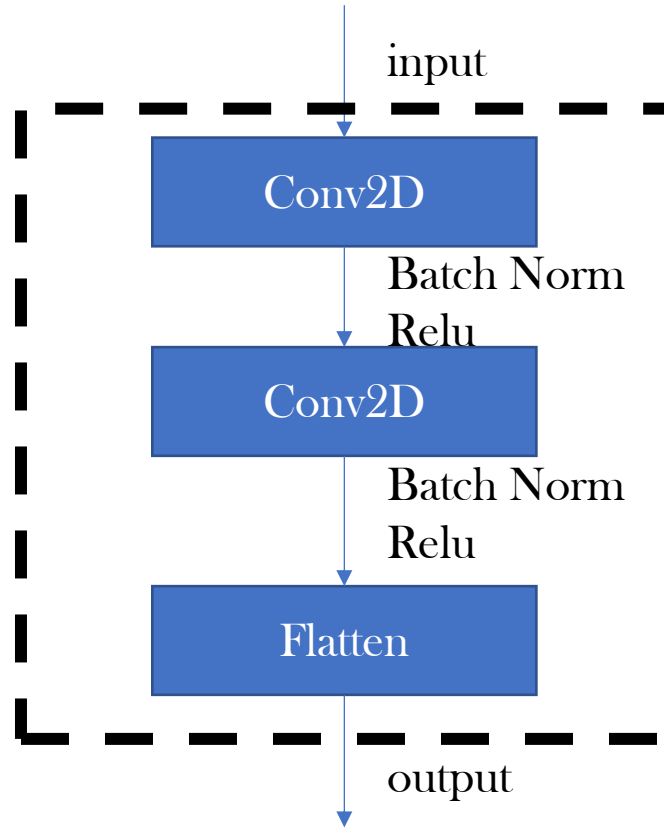
        self.conv2d1 = nn.Conv2d(in_channels=self.N, out_channels=32,
                                   kernel_size = (1,self.L), stride = 1, padding = "valid", bias=False)
        self.batchnorm1 = nn.BatchNorm2d(32)
        self.relu = nn.ReLU()

        self.conv2d2 = nn.Conv2d(in_channels=32, out_channels=1,
                                   kernel_size = (1,1), stride = 1, padding = "valid", bias=False)
        self.batchnorm2 = nn.BatchNorm2d(1)
        self.flatten = nn.Flatten()

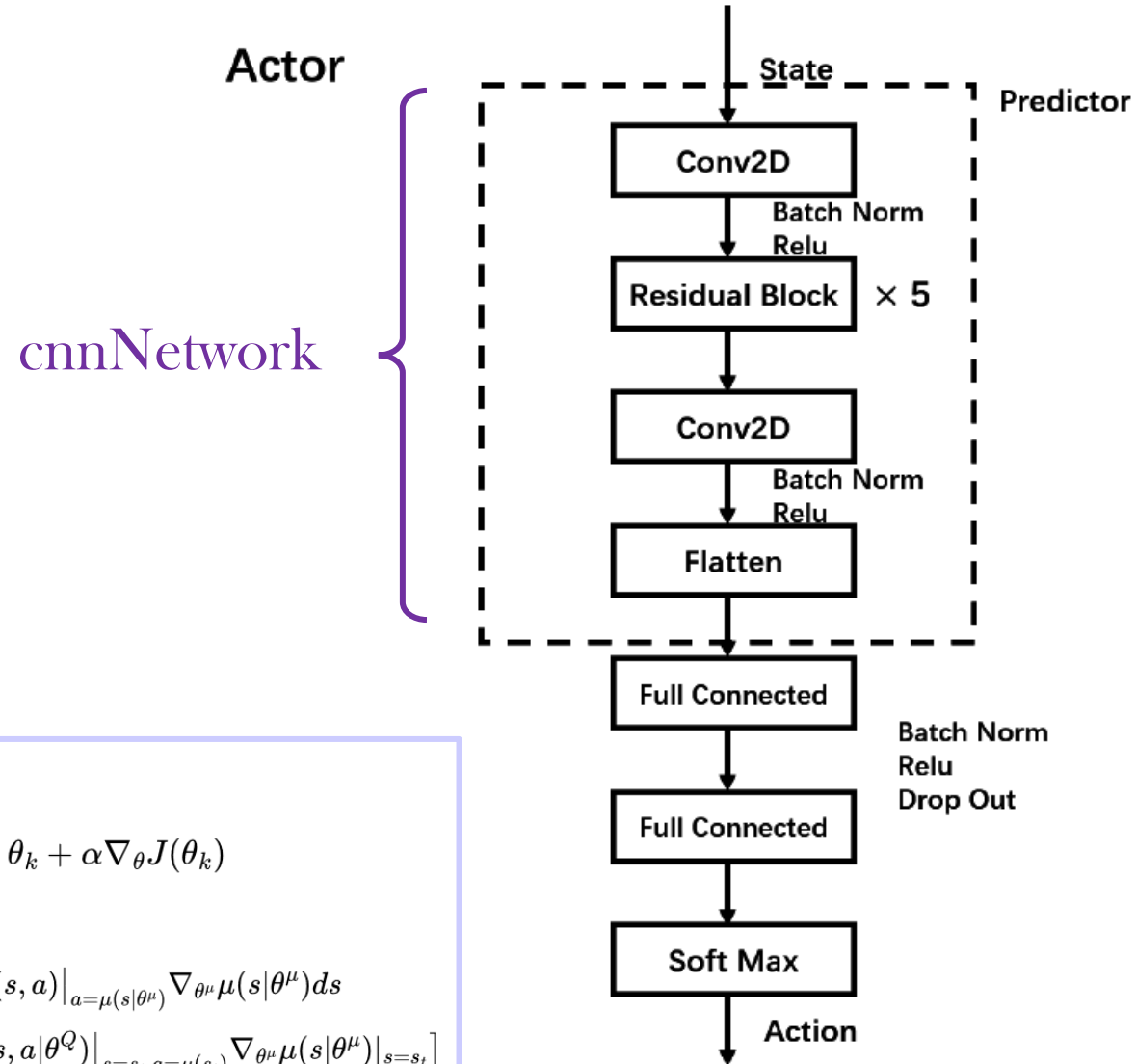
    def forward(self, x):
        y = self.conv2d1(x)
        y = self.relu(self.batchnorm1(y))
        y = self.conv2d2(y)
        y = self.relu(self.batchnorm2(y))
        y = self.flatten(y)
        return y
```

Code (network)

- cnnNetwork



Code (network)



The update scheme is as below:

$$\theta_{k+1} = \theta_k + \alpha \nabla_{\theta} J(\theta_k)$$

where

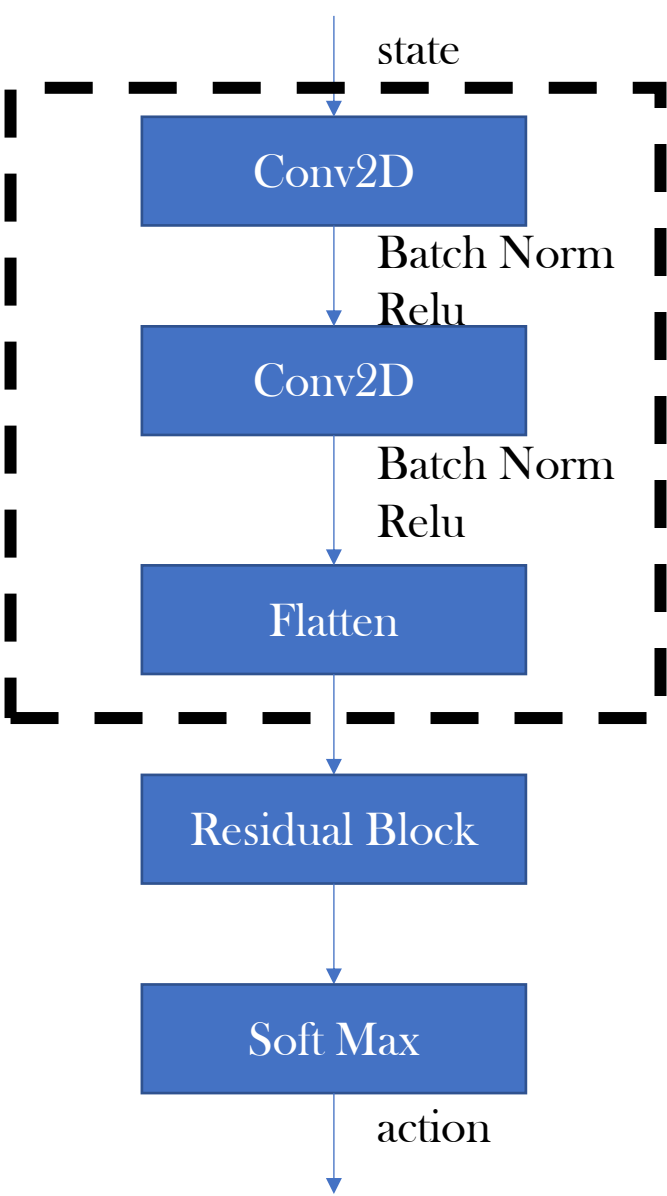
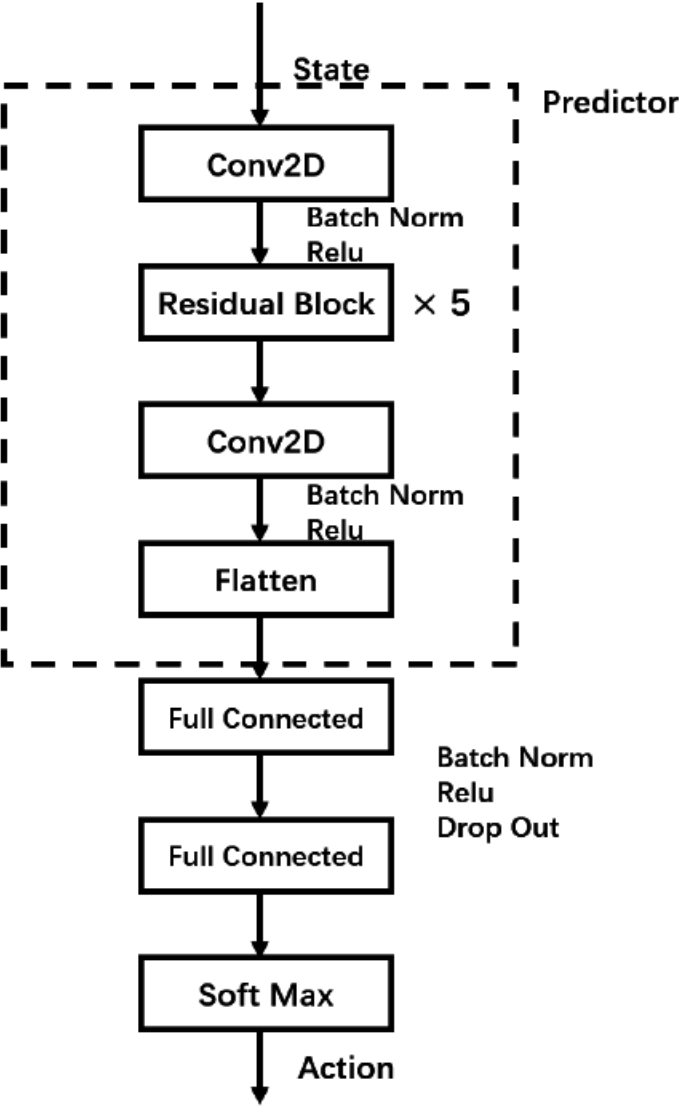
$$\begin{aligned} \nabla_{\theta} J(\theta) &= \int_S \rho^{\mu}(s) \nabla_a Q(s, a) \big|_{a=\mu(s|\theta^{\mu})} \nabla_{\theta^{\mu}} \mu(s|\theta^{\mu}) ds \\ &= \mathbb{E}_{s_t \sim \rho^{\beta}} \left[\nabla_a Q(s, a|\theta^Q) \big|_{s=s_t, a=\mu(s_t)} \nabla_{\theta^{\mu}} \mu(s|\theta^{\mu}) \big|_{s=s_t} \right] \end{aligned}$$

Code (network)

```
class DDPGActor(nn.Module):
    def __init__(self, M, L, N): # pass number of assets (M) for defining the network
        super(DDPGActor, self).__init__()
        self.M = M
        self.L = L
        self.N = N
        self.basic = cnnNetwork(self.M, self.L, self.N)
        self.fcl = nn.Linear(self.M, self.M)
        self.softmax = nn.Softmax()

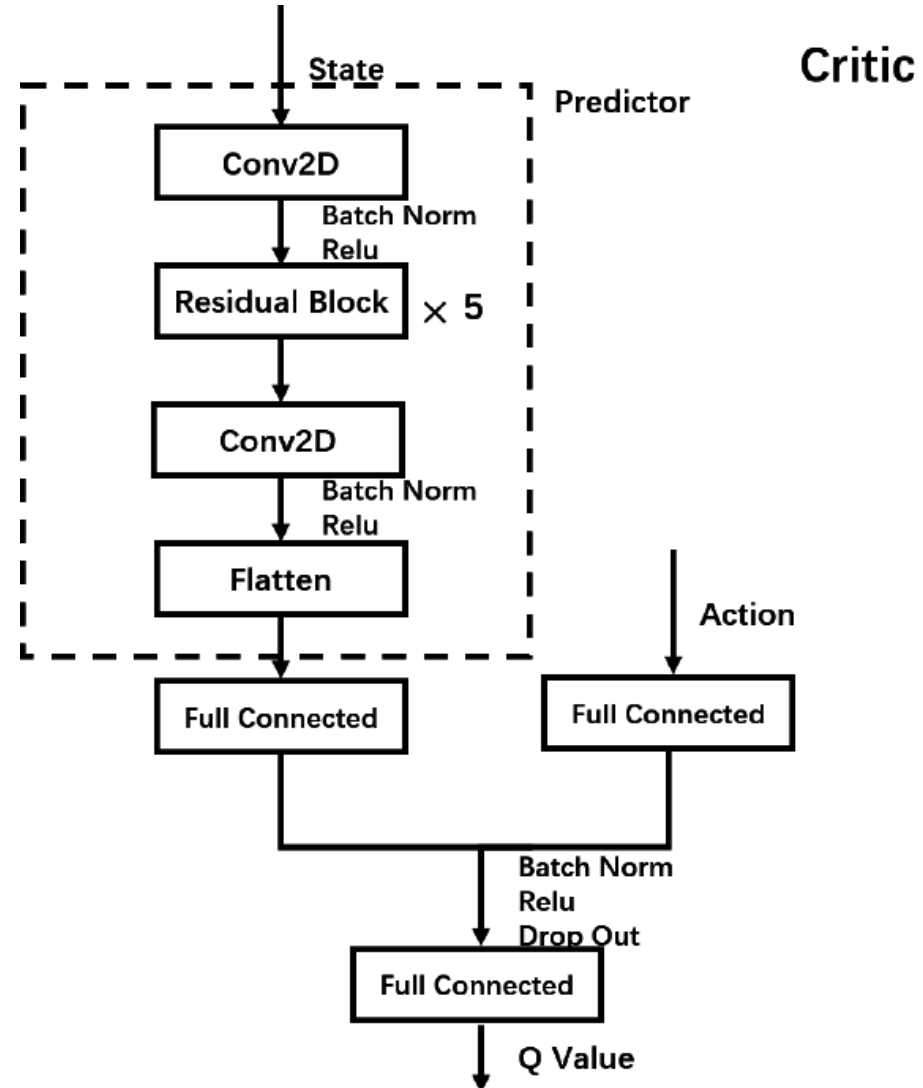
    def forward(self, input):
        x, w = input
        # x = x.permute(0, 3, 1, 2)
        y = self.basic(x)
        y = self.fcl(torch.add(y, w)) ← Residual Block
        y = self.softmax(y)
        return y
```

Code (network)



Code (network)

$$\mu(s) = \operatorname{argmax}_a Q(s, a)$$



Code (network)

```
class DDPGCritic(nn.Module):
    def __init__(self, M, L, N):
        super(DDPGCritic, self).__init__()
        self.M = M
        self.L = L
        self.N = N
        self.basic = cnnNetwork(self.M, self.L, self.N)
        self.fc1 = nn.Linear(self.M, 1)

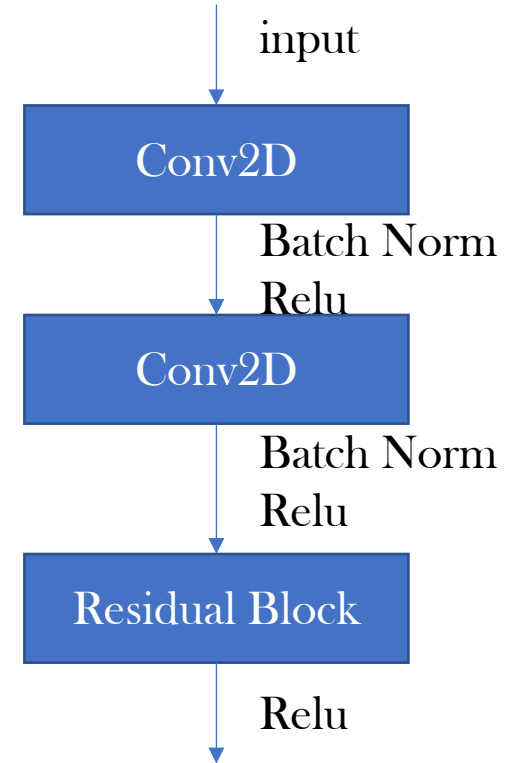
    def forward(self, input):
        x, w , action = input
        # x = x.permute(0, 3, 1, 2)
        y = self.basic(x)
        y = torch.add(y, action)
        y = torch.add(y, w) ← Residual Block
        y = self.fc1(y)
        return y
```

Code (network)

```
import torch
import torch.nn as nn
class resblock(nn.Module):
    """ Resnet block """

    def __init__(self, n_channels):
        super(resblock, self).__init__()
        self.conv1 = nn.Conv2d(in_channels = n_channels, out_channels = n_channels,
                                kernel_size = 3, stride = 1, padding = 1, bias = False)
        self.batchnorm1 = nn.BatchNorm2d(n_channels)
        self.relu = nn.ReLU()
        self.conv2 = nn.Conv2d(in_channels = n_channels, out_channels = n_channels,
                                kernel_size = 3, stride = 1, padding = 1, bias = False)
        self.batchnorm2 = nn.BatchNorm2d(n_channels)

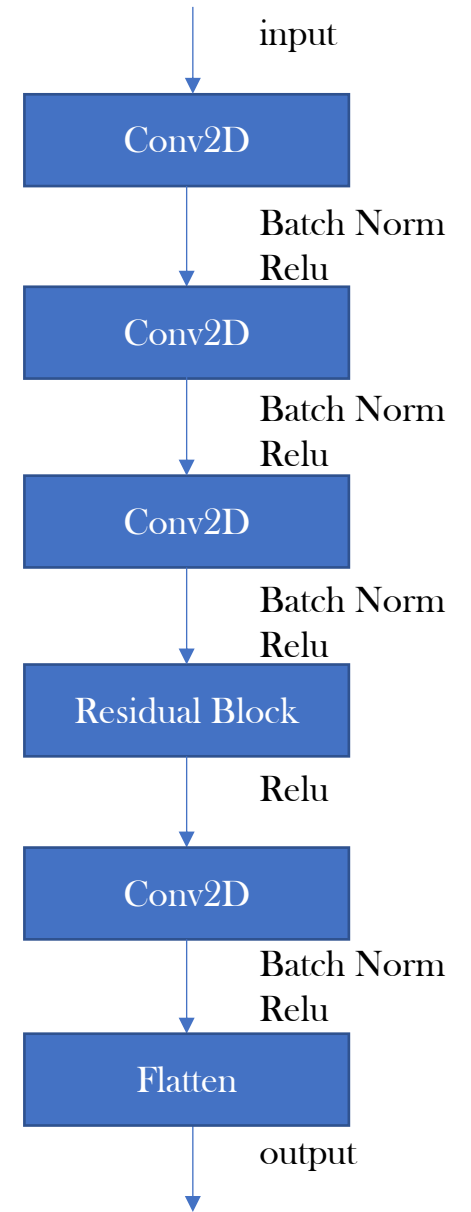
    def forward(self, x):
        y = self.relu(self.batchnorm1(self.conv1(x)))
        y = self.relu(x + self.batchnorm2(self.conv2(y)))
        return y
```



Code (network)

```
class basicNetwork(nn.Module):
    def __init__(self):
        super(basicNetwork, self).__init__()
        self.n_channels = 7
        self.conv1 = nn.Conv2d(in_channels = self.n_channels, out_channels = self.n_channels,
                                kernel_size = 1, stride = 1, padding = 0, bias = False).float()
        self.res1 = resblock(self.n_channels).double()
        self.batchnorm1 = nn.BatchNorm2d(self.n_channels)
        self.relu = nn.ReLU()
        self.conv2 = nn.Conv2d(in_channels = self.n_channels, out_channels = self.n_channels,
                                kernel_size = 1, stride = 1, padding = 0, bias = False)
        self.batchnorm2 = nn.BatchNorm2d(self.n_channels)
        self.flatten = nn.Flatten()

    def forward(self, x):
        y = self.conv1(x)
        y = self.relu(self.batchnorm1(y))
        y = self.res1(y)
        y = self.relu(self.batchnorm2(self.conv2(y)))
        y = self.flatten(y)
        return y
```



Code

1. network
2. ddpg

Code (ddpg)

```
class DDPG():
    def __init__(self, M, L, N):
        # Hyperparameters of state
        self.M = M # Number of assets = m (stock/illiquid) + 1 (cash/liquid) {M includes cash here} [dflt: M=7]
        self.L = L # Length of time-series used in one state [dflt: L=10]
        self.N = N # Number of features to use {ex: close, high} [dflt: N=5]

        # Buffer initialization
        self.s_buffer = []
        self.a_buffer = []
        self.r_buffer = []
        self.isnt_buffer = []
        self.s_next_buffer = []
        self.w_buffer = []

        # Hyper-parameters for training
        self.tau = 10e-2 # Tau for soft-update
        self.batch_size = 1
        self.discount = 0.9 # Discount for future rewards
        self.lr_actor = 3e-5
        self.lr_critic = 1e-5

        self.buffer = list() # initializes empty list as buffer
```

Code (ddpg)

```
# Initializing models (as double to prevent casting issues)
self.actor = DDPGActor(self.M, self.L, self.N).double()
self.actor_target = DDPGActor(self.M, self.L, self.N).double()
self.actor_optim = Adam(self.actor.parameters(), lr=self.lr_actor)

self.critic = DDPGCritic(self.M, self.L, self.N).double()
self.critic_target = DDPGCritic(self.M, self.L, self.N).double()
self.critic_optim = Adam(self.critic.parameters(), lr=self.lr_critic)

# hard updating sets parameters of target/offline model equal
hard_update(self.actor_target, self.actor)
hard_update(self.critic_target, self.critic)

self.criterion = nn.MSELoss() # loss function choice
```

$$\theta^{Q'} \leftarrow \theta^Q, \theta^{\mu'} \leftarrow \theta^{\mu}$$

Code (ddpg)

```
def predict(self, s, action):  
    return self.actor((torch.tensor(s).double().permute(0, 3, 1, 2), action.double())) # needs to be passed as a tuple  
  
def save_transition(self, state, action, r, is_nonterminal, next_state, prev_action):  
    # self.buffer.append((state, action, r, is_nonterminal, next_state, prev_action))  
    self.s_buffer.append(state)  
    self.a_buffer.append(action)  
    self.r_buffer.append(r)  
    self.isnt_buffer.append(is_nonterminal)  
    self.s_next_buffer.append(next_state)  
    self.w_buffer.append(prev_action)
```

Code (ddpg)

```
def train(self):
    s, a, r, isnt, s_next, w = torch.tensor(self.s_buffer).permute(0, 1, 4, 2, 3).squeeze(1),
    torch.stack(self.a_buffer).squeeze(1), torch.tensor(self.r_buffer).unsqueeze(1), torch.tensor(self.isnt_buffer).unsqueeze(1),
    torch.tensor(self.s_next_buffer).permute(0, 1, 4, 2, 3).squeeze(1), torch.stack(self.w_buffer).squeeze(1)

    length = len(s_next)

    y = r + self.discount*isnt*self.critic_target( (s_next, a.detach(), self.actor_target((s_next, a.detach())) ) )

    # updating critic
    self.critic_optim.zero_grad()
    q_pred = self.critic( (s, w.detach(), a.detach()) )
    c_loss = self.criterion(q_pred, y)
    c_loss.backward()
    self.critic_optim.step()

    # updating actor
    self.actor_optim.zero_grad()
    a_loss = -self.critic( (s, w.detach(), self.actor((s, w.detach())) ) ).mean()
    a_loss.backward()
    self.actor_optim.step()

    # soft updating targets
    soft_update(self.actor_target, self.actor, self.tau)
    soft_update(self.critic_target, self.critic, self.tau)
```

Code (ddpg)

```
def get_batch(self):
    s = [torch.tensor(data[0]).double() for data in self.buffer]
    a = [data[1].double() for data in self.buffer]
    r = [torch.tensor(data[2]).double() for data in self.buffer]
    isnt = [torch.tensor(data[3]).double() for data in self.buffer]
    s_next = [torch.tensor(data[4]).double() for data in self.buffer]
    w = [data[5].double() for data in self.buffer]
    return s, a, r, isnt, s_next, w

def reset_buffer(self):
    self.s_buffer = []
    self.a_buffer = []
    self.r_buffer = []
    self.isnt_buffer = []
    self.s_next_buffer = []
    self.w_buffer = []
```

Thank you for listening